Sovereign Software

Governing the Full Substrate Through Constitutional Computing

Neither an attestation nor a complete inventory confers sovereignty by itself.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Possessing application source does not by itself establish control over the system that gives the application its behavior. Dependencies, build tools, runtimes, identity services, update channels, remote infrastructure, and hardware can retain powers that the nominal owner neither understands nor governs. This paper develops **sovereign software** as a relational and operational property: the ability of identified people or institutions to determine software meaning, constrain effects, inspect the grounds of assurance, and continue, repair, or replace required capabilities without ungranted control by an indispensable supplier. As a companion to *Constitutional Computing: Evidence-Carrying Realization, Agentic Development, and the Economics of Structured Abundance*, it presents sovereignty as the purpose that structured abundance can help make practical. It distinguishes legal freedom, source access, custody, security, and effective independence; develops a thirteen-axis custody profile and a claim-relative dependency-closure model; and treats reconstruction and provider-removal exercises as tests of meaningful exit. Constitutional contracts connect these requirements to independent evidence, locally governed adoption, and competing implementations. In universal-GSM terms, the paper follows sovereignty from possibility and specification through admissible and preferred configurations, a selected instance constitution/configuration, its instance/implementation realization, evidence, and revision. Agentic development and distributed publication are proposed as ways to turn opaque dependencies into bounded reliance relationships and, where feasible, replaceable realizations. Neither an attestation nor a complete inventory confers sovereignty by itself. The paper examines build bootstrap, information-flow limits, version sovereignty, closed-source adaptation, shared governance, and the risks of creating new gatekeepers around models or verification. An encrypted-messaging scenario and a falsifiable evaluation program specify how the approach could be investigated without claiming universal verification, zero residual trust, or a mature sovereign operating system.

Keywords: sovereign software; Constitutional Computing; software freedom; dependency custody; local-first software; reproducible builds; execution authority; effective exit; structured abundance.

1. Motivation: owning an application without owning its conditions

The research program developed here began with an application rather than an operating-system theory. The author spent approximately a month using Replit to construct an encrypted chat application. Working through React, Node.js, and a broader dependency and hosting environment exposed a distinction between implementing application features and establishing control over the machinery on which those features depended. Questions about opaque libraries, transitive dependencies, possible telemetry, and external operational control shifted the objective from completing one application to pursuing software that could be governed throughout its substrate.

This account is autobiographical motivation, not a forensic finding about a particular Replit deployment, React, or Node.js. No original dependency tree or traffic capture is analyzed here. The presence of a framework does not establish telemetry, and a language or runtime name is not an assurance category. The relevant discovery was a lack of sufficient knowledge and control, rather than a demonstrated malicious property of those named technologies.

The resulting question was nevertheless substantive: **what does it mean to own an application when other actors can determine its dependencies, change its execution conditions, observe its operation, or make continued use conditional on their participation?** Encryption addresses specific confidentiality and authentication objectives; it does not determine every aspect of endpoint execution, metadata exposure, identity administration, or lifecycle control. Messaging Layer Security, for example, expressly separates cryptographic protection from assumptions about authentication, delivery, and observable metadata [10].

A source repository is consequently only one part of the object requiring governance. Software reaches the user through a construction history and operates through a dependency structure. The scope includes what the software means, what built it, what can alter it, which services it requires, whose credentials matter, which observations leave the machine, and whether a successor can reconstruct the required capability. The aspiration to be "sovereign all the way down" is a demand to keep following those relationships rather than permitting abstractions to conceal authority.

The earlier companion paper examines how model-centered development, competing realizations, and shared qualification evidence could produce structured abundance [1]. This paper asks the complementary question: **what must remain under users' effective control for that abundance to advance sovereignty rather than create a more efficient form of dependence?**

Cheap implementations are not sufficient if adoption, interpretation, or verification is monopolized elsewhere.

The contributions are a definition of sovereignty relative to actors and claims; an operational framework grounded in the existing SOV-0 custody model; a recursive account of substrate governance and its trust boundaries; and an evaluation agenda centered on continuation, substitution, and independent reconstruction. This is an architectural and normative research-position paper. It reports no newly executed security audit, migration, benchmark, or user study.

2. Related traditions and the scope of the contribution

2.1 Software freedom is a foundation, not a straw opponent

The free-software tradition already centers users' individual and collective control through freedoms to run, study, modify, and redistribute software. The Open Source Definition likewise requires more than source visibility: it includes distribution and modification rights [7,8]. Sovereign software must not claim to discover user agency by inaccurately reducing these traditions to permission to read files.

The additional engineering question is how usable freedom is embodied across the full lifecycle. A person may possess source and modification rights while lacking a reconstructable build environment, usable keys, migration knowledge, or the resources needed to maintain a fork. Rights remain essential; practical exercises determine whether those rights can be enacted for a particular capability. Constitutional Computing is proposed as an enabling architecture, not a replacement definition of free or open-source software.

2.2 Local-first computing and situated sovereignty

Local-first software connects collaboration with user-held data, offline operation, longevity, and agency. Its architectural relocation of the primary data copy is an important precedent [9]. This paper extends the inquiry from where working data resides to who governs models, dependencies, compilation, verification, updates, and exit. Local storage is valuable, but a locally installed binary can still depend on a remote identity service or an unavailable reconstruction process.

Digital sovereignty also has state-centered, organizational, and individual meanings. Pohle and Thiel examine the concept's contested political uses [11]. Here the subject is explicitly identified: a person, association, institution, or jointly governed group exercising legitimate control over specified computational activities. National location, domestic ownership, and individual autonomy are not assumed to coincide. Moving a dependency to a domestic provider does not, by itself, establish users' ability to refuse or replace it.

2.3 Protection, reproducibility, and supply-chain assurance

Saltzer and Schroeder supply longstanding principles of protection, including least privilege and complete mediation; Capsicum offers a concrete capability-based approach to constraining programs' access [12,13]. Reproducible builds, Nix, and in-toto address distinct aspects of construction, dependency identity, deployment, and supply-chain evidence [14–16]. The Update Framework addresses attacks against update distribution [17]. These mechanisms are predecessors and potential components, not rival explanations to be dismissed.

The remaining integration problem is that their guarantees address different questions. A reproducible binary can faithfully reproduce undesirable behavior. An authentic update can contain a change the user does not wish to adopt. A sandbox can constrain an opaque component without revealing its internals. The contribution proposed here is a common account of what each mechanism establishes, who controls the relevant decision, and which dependency remains indispensable.

Compiler bootstrap and hardware assurance further prevent the inquiry from stopping at application code. Diverse double-compiling investigates source-to-executable correspondence under stated assumptions; bootstrappable-build efforts address the provenance and size of trusted starting points [18,19]. seL4 and CHERI demonstrate substantial work on formally specified operating-system behavior, memory protection, and compartmentalization while retaining explicit proof and hardware boundaries [20,21]. Sovereignty neither originates these techniques nor follows automatically from adopting them.

3. Defining sovereign software

3.1 A relation, not a product label

For this paper, sovereign software is software whose relevant meaning, custody, effects, evolution, and continuation can be governed by identified principals under an explicit jurisdiction, with evidence adequate to the claims made and without an ungranted indispensable external controller. This definition is relative to an operation set, environment, resource envelope, and time. It is not an assertion that every person can command every component or that no outside dependency exists.

Five distinctions are necessary. **Legal permission** concerns rights and applicable restrictions. **Custody** concerns possession and practical administration of artifacts, data, or keys. **Authority** concerns who is legitimately permitted to decide or act. **Assurance** concerns what evidence supports a property. **Effective independence** concerns whether required activity can continue or move when a particular dependency is withdrawn. These dimensions can diverge.

A secure hosted service may provide strong confidentiality while leaving a customer unable to operate without the provider. An independently operated system may offer substantial control while containing serious defects. Security

and sovereignty reinforce each other when unauthorized influence is constrained, but neither subsumes the other. A documented trust relationship is more legible than an undocumented one; documentation alone does not eliminate the dependence.

"Ownership" is therefore used operationally, not as a blanket legal assertion over every component. The intended subject owns or legitimately governs its data, commitments, decisions, and authorized operations while relying on separately licensed implementations and shared institutions. A model describing another person's data does not create permission to access it.

3.2 The thirteen custody dimensions

SOV-0 already defines thirteen axes, componentwise comparisons, separately typed evidence, and claim-specific ceilings [2]. This paper adopts that decomposition rather than introducing a scalar sovereignty index. The following questions operationalize the axes without claiming to replace their controlling schema.

Custody dimension	Operational question
Canonical-model custody	Can the principal retain and revise the authoritative model and operation definitions?
Data custody	Can required state, history, and metadata be held, interpreted, protected, and recovered?
Source custody	Is the relevant preferred source form accessible and usable under applicable rights?
Dependency custody	Are direct and transitive materials identified, retained, and replaceable where required?
Build custody	Can the required artifacts be constructed without an indispensable provider build service?
Toolchain custody	Are compilers, generators, linkers, and their bootstrap dependencies governed?
Runtime custody	Who can execute, observe, interrupt, alter, or bypass the relevant enforcement boundary?
Identity and key custody	Who can authenticate, recover, rotate, delegate, or revoke the necessary identities and keys?
Update authority	Who decides which code, policies, schemas, and contracts become active?

Custody dimension	Operational question
Verification custody	Can evidence be checked or challenged without one indispensable verifier's permission?
Observability and audit	Can effects and deviations be witnessed without mandatory external disclosure?
Evolution and fork custody	Can the principal maintain or commission a successor without the incumbent's cooperation?
Exit and reconstruction custody	Can required capability and state be restored independently, not merely exported?

The axes do not form a universal ladder. A managed service can be stronger on recovery assistance while weaker on runtime custody; a local implementation can invert that relationship. Evidence categories also describe methods and scope, not an automatic ordering in which any third-party signature outranks every local test.

A profile may be written schematically as:

$$\text{Profile}(u, X, \Omega, t) = \{(a_i, c_i, r_i, e_i, l_i, f_i)\}.$$

Here u identifies the governing principal, X the relevant system, Ω the operating and claim profile, and t the evaluation point. A claim is supportable only if all its required dimensions satisfy the selected criteria. Good data custody cannot numerically compensate for a missing dependency that makes independent reconstruction impossible.

3.3 Effective choice and non-domination

A practical sovereignty claim should name a decision the principal can exercise: keeping a version, replacing a relay, changing a realization, withholding telemetry, checking a proof, or continuing after provider withdrawal. A nominal choice whose exercise requires unaffordable reconstruction is not equivalent to a regularly tested alternative.

This account is compatible with delegation. A user can appoint a maintainer to install routine patches or an institution can require multi-party approval for consequential changes. The delegation should have inspectable scope, recovery conditions, and a feasible path to transfer responsibility. Sovereignty does not require clicking approval on every event; it requires that automation operate under legitimate, revisable mandates rather than its own expansion of power.

4. Governing the dependency closure

4.1 The substrate is a graph

The full substrate is not a simple vertical stack. A program may depend on a package registry for construction, a certificate authority for communication, a remote model for maintenance, a driver for input, and a verifier for release admission. Each relationship can introduce distinct authority or availability conditions.

Represent the relevant environment as a typed graph:

$$G = (V, E_{use} \cup E_{build} \cup E_{govern} \cup E_{evidence} \cup E_{reconstruct}).$$

Nodes include artifacts, services, semantic contracts, principals, policies, hardware profiles, and evidence. Edges identify how a node uses, produces, governs, justifies, or reconstructs another. A claim-relative closure follows every relationship whose failure or alteration can affect that claim, using explicit extraction and coverage rules.

Software bills of materials provide useful machine-readable inventory conventions, including the SPDX ecosystem [22]. The proposed closure is broader than a package list. It also asks who can change a dependency, which runtime channels remain available, what evidence supports its semantics, and what an adopter needs after the original provider disappears.

4.2 Inventory is not control

For each critical node, a reliance record should identify exact material or service identity, role, controller, accessible interfaces, permitted effects, evidence basis, residual assumptions, and exit options. A versioned service declaration may still leave the remotely executed implementation unknown. A hash identifies a retained object, not the continuing honesty of its publisher or its compatibility with a new environment.

The graph must itself be challenged. Runtime tracing can discover undeclared accesses, but an unobserved access is not thereby impossible. Build instrumentation can identify sampled dependencies, but a conditional download may remain dormant. Source analysis, constrained execution, adversarial tests, and independent extraction can provide complementary coverage. Unknown closure completeness must remain unknown for claims that require completeness.

An opaque dependency is not an exception to be hidden; it is a node with a limited claim ceiling. But merely labeling it opaque does not establish sovereignty. If it can unilaterally disable a required capability and no qualified continuation exists, that dependence remains operationally consequential. Legibility is the beginning of governance, not its completion.

4.3 The enforcement condition

Suppose a sovereignty claim requires every in-scope network disclosure to pass through a local gate. If a dependency has another reachable path that can disclose the same data without that gate, the claim fails. This is an application of complete mediation, not a new security theorem [12]. It explains why wrapping a high-level tool API may be inadequate when a process also retains sockets, shell access, or inherited credentials.

No diagram can make the underlying operating system incapable of bypass. The enforcement mechanism, boot path, privileged administrators, and relevant hardware assumptions belong inside the claim's accounting. A useful migration can progressively replace or constrain these elements. It may not relabel a previously unexamined lower layer as sovereign merely because the application above it acquired a constitution.

5. Constitutional control across meaning and execution

5.1 Authority follows the operation, not the implementation

The companion architecture separates domain models, operation contracts, candidate realizations, evidence, dependency history, and authority [1]. GOML, GSOL, and MCIR distinguish object/state meaning, admissible transitions, and computational realization. Their relationships are governed correspondences, not an obligatory hierarchy [5].

For sovereignty, the consequential property is that replacing a realization need not surrender the operation's governing meaning. A locally held messaging contract can outlive one relay implementation. A business can maintain its accounting meanings while commissioning another storage backend. An implementation remains authoritative about its actual mechanics, but does not acquire the right to redefine the user's commitments through an undocumented schema change.

GSOL supplies a specification surface for preconditions, effects, refusals, evidence obligations, claims, nonclaims, and permitted changes in standing. It does not supply authority simply by asserting that an operation is allowed. A separately governed execution boundary must validate the relevant grant against the target, parameters, active policy, and current state [4].

The distinction can be stated simply:

$$\begin{aligned} \text{proof of a property} &\neq \text{authority to execute},\\ \text{authenticated publisher} &\neq \text{local adoption authority},\\ \text{effect observed} &\neq \text{intended outcome established}. \end{aligned}$$

These are independent obligations, not alternatives. A trustworthy implementation can still be invoked by the wrong principal, against the wrong target, or after authorization has changed.

5.2 Public rules need not expose private state

Constitutional governance should permit public semantic contracts and independently buildable checkers while keeping private source, working data, credentials, and local decisions under their legitimate custodians. The Cognitive ABI separates capability description from grants and makes qualification specific to an environment and evidence profile [6].

A user can request outside checking of a redacted artifact, but withholding required material limits what the checker can establish. Selective disclosure must neither fabricate a complete proof nor leak sensitive identifiers merely to demonstrate that a premise exists. Public inspectability of the framework is compatible with private execution and evidence custody.

5.3 The governing machinery is part of the substrate

Policy engines, parsers, canonicalizers, proof checkers, schedulers, model registries, and agent harnesses can themselves become indispensable dependencies. They need the same identity, source, build, effect, update, and exit treatment as the applications they govern. A system that exposes application code while placing its constitutional interpreter behind a mandatory remote service has moved the dependency rather than resolved it.

Self-application can reveal gaps, but it cannot bootstrap its own independent authority. TCLA explicitly separates material-backed evidence from ratification and requires separately established schemas, translation identities, and proof assumptions [23]. A checker can evaluate a candidate replacement without granting that replacement its own standing. Small trusted kernels, diverse witnesses, and retained evidence make reliance more examinable; no component becomes infallible by being foundational.

6. Source, construction, runtime, and information flow

6.1 From source custody to independent construction

A sovereign build profile should retain relevant source, exact dependency materials, generators, compiler and linker identities, instructions, options, and environment assumptions. It should distinguish acquisition from construction. A build claimed to operate offline cannot silently acquire a new package merely because its lockfile names one.

Reproducible-build practice establishes whether specified artifacts can be recreated identically from corresponding source, instructions, and environment [14]. Nix supplies a longstanding design for distinguishing dependency-sensitive component instances [15]. These are valuable mechanisms for an independently reconstructable closure, but retained identity and availability are different: knowing a missing archive's hash does not recover its bytes.

Source-to-binary correspondence also has a bootstrap question. Rebuilding

with the same compromised compiler may reproduce the compromise. Diverse double-compiling addresses a particular correspondence problem under explicit assumptions about a trusted diverse toolchain; bootstrappable-build work addresses trusted seeds and their construction histories [18,19]. Such work belongs to the sovereignty program rather than being dismissed as an unreachable layer.

A verified compiler establishes only its supported theorem. CompCert, for example, states semantic preservation for an identified compilation path and observation model; its documentation distinguishes external construction stages and excludes execution time and memory consumption from the ordinary observable-behavior theorem [24]. Runtime cost limits or complete source-to-installed-image correspondence therefore need additional evidence.

6.2 Source availability and safe dependency reuse

The goal is not dependency avoidance at any cost. Rewriting a mature cryptographic primitive without adequate expertise can reduce assurance while increasing apparent authorship. A dependency can be sovereign-compatible when its role, material, permitted effects, qualification, maintenance, and substitution conditions are adequately governed.

Conversely, an in-house component can be non-sovereign in practice if one employee alone can build it or if it requires a private service whose credentials cannot be recovered. The question is not who first wrote the code, but whether legitimate successors can understand, operate, maintain, and replace the required capability under the declared conditions.

6.3 Telemetry, audit, and permitted communication

The originating telemetry concern becomes an effect contract. A no-telemetry profile should specify prohibited data classes, destinations, background tasks, update lookups, crash uploads, and diagnostic transmission. Required communications should be separately identified. For a chat application, transmitting user-authorized messages is an intended effect; sending diagnostics to a third-party endpoint is another operation, not an incidental entitlement of a library.

Network allowlisting alone is insufficient. A compromised component permitted to contact a relay may misuse that permitted channel. Containment must account for which secrets the component can read, which content it can emit, and what release rules apply. Capability separation, narrow interfaces, independent checking, and appropriate information-flow reasoning can reduce this risk. Claims about all covert or physical side channels require their own scope rather than following from a list of allowed hosts.

Observability must not recreate the surveillance being avoided. Local logs should minimize sensitive content, specify retention and access, and distinguish private evidence from material approved for publication. A receipt containing plaintext

messages would itself create a new disclosure obligation. An audit mechanism's value does not authorize its data collection.

6.4 Security boundaries do not end the recursion

Capsicum demonstrates practical restriction of ambient process authority; CHERI supplies hardware-assisted memory protection and compartmentalization mechanisms [13,21]. seL4 demonstrates strong machine-checked properties while documenting assumptions involving boot code, hardware, and other interfaces [20]. These approaches can strengthen a sovereign substrate, but a product name is not a substitute for selecting and qualifying a concrete configuration.

Firmware, device behavior, browser extensions, privileged host services, and physical access can remain critical. Their presence is not a reason to abandon the program. It is a reason to state which claims are established now and which require the next observation, model, implementation, or enforcement boundary.

7. Version sovereignty, freshness, and reversible change

Version sovereignty is control over adoption, not an assertion that an old release remains secure forever. A user should be able to retain a permitted installed realization, inspect its known limitations, and decide whether to update, isolate, repair, or replace it. A vendor's support expiry must be distinguishable from a technical requirement for the retained application to stop working.

Secure update distribution and adoption authority are complementary. TUF distinguishes authenticated, sufficiently fresh update information from application-specific update decisions [17]. A constitutional installation can use such mechanisms while retaining a locally selected policy for which candidate becomes active. Provider authenticity does not make a new contract, telemetry policy, or dependency acceptable automatically.

This distinction also separates a malicious rollback or freeze from an informed decision to retain older code. The former misrepresents what is available or authentic; the latter records a deliberate local choice with the relevant evidence. Supporting intentional retention does not require disabling integrity checks or accepting stale metadata as current. Newly required network interoperability and external service availability must be disclosed rather than promised indefinitely.

Offline operation imposes an epistemic limit: a node cannot know about a revocation it has not received. Receipts should identify the last observed authority or evidence snapshot and the policy governing that interval. Local policy may restrict consequential operations when required freshness cannot be established. That decision should arise from the user's adopted requirements, not a hidden vendor kill switch.

Rollback also requires more than keeping an old executable. State migrations, keys, and externally visible actions may not be reversible. Proposed changes should carry compatible checkpoints, migration relations, and explicit irreversible

consequences. An emergency policy can authorize rapid action within pre-established bounds, but emergency language cannot silently enlarge those bounds.

8. Effective exit: from export to reconstruction

8.1 Export is an intermediate artifact

An export can preserve records while losing the ability to interpret or operate on them. Continuing an application may require schema definitions, attachment references, provenance, access rules, identity relationships, conflict semantics, and executable migration logic. A collection of files is not equivalent to a functioning successor. SOV-0 explicitly requires reconstruction contracts and restore exercises rather than accepting vendor export as independence [2].

Define an exit contract for principal u, required capability k, incumbent dependency set V, and resource envelope b. It identifies which observations and future operations must survive removal, which changes are permitted, and which resources can be used. A successful exit witness includes retained inputs, an identified successor, a migration record, checks against the preservation relation, and successful continuation without services in V.

The criterion is not necessarily byte identity. A successor may use different storage or algorithms while preserving the relevant meaning. Equally, identical visible records may conceal a loss of permissions or audit history. The exit relation must say what matters before migration is evaluated.

8.2 Provider-removal exercises

A meaningful exercise makes the incumbent unavailable in a controlled environment. Provider credentials, hosted build services, registry access, and identity callbacks are withheld where they fall within the tested dependency set. A separate operator attempts restoration using only the retained package and the declared successor resources.

The result should disclose elapsed time, human intervention, compute and storage use, unrecovered state, remaining supplier calls, and failed semantic checks. A claim established on one workload is not generalized silently to all workloads. A negative result is valuable: it locates the real barrier to exit rather than preserving the appearance of portability.

Exit cost can be decomposed for measurement:

$$C_{exit} = C_{material} + C_{interpretation} + C_{rebuild} + C_{migration} + C_{requalification} + C_{coordination}.$$

These terms are not a sovereignty score. They measure the practical expense of a specified transition. Rights may permit a fork while the required expertise makes this cost prohibitive. Shared models, retained toolchains, agents, and

reusable evidence can lower particular terms; that is an empirical hypothesis, not an automatic consequence of publication.

8.3 Independence includes correlated alternatives

Five implementations that all require the same unavailable identity provider do not supply five independent exit paths. Substitutes should be compared through their critical shared dependencies. A supplier that lies on every feasible continuation path remains an effective point of control, even when the application marketplace appears diverse.

This suggests testing removal of dependency groups as well as individual vendors. Reconstruction may survive loss of one repository while failing after loss of a shared signing service or compiler download. The relevant unit is the capability's dependency structure, not brand count.

Keys require particular care. Some profiles intentionally use non-exportable hardware keys. An exit plan may preserve control through supported signing, authorized rotation, or an explicitly authenticated identity transition rather than raw key extraction. The plan must neither invent recovery authority nor weaken forward-secrecy requirements by retaining secrets that the protocol requires to be erased.

9. Opaque and closed-source systems as bounded reliance

A sovereign direction need not begin with immediate replacement of every opaque component. An organization can first retain its own operational model, constrain the vendor interface, independently record relevant effects, and test an exit path. SOV-0 models precisely this distinction: local control over an opaque platform's observable use does not establish internal platform verification [2].

The wrapper can admit a specific operation before invocation, validate observable outcomes, and preserve limitations. It must not describe an irreversible remote disclosure as something a later refusal undoes. Once data reaches an external system, a local receipt cannot erase remote copies. Post-effect checking supports reconciliation, future restrictions, or compensation where possible; it is not a substitute for pre-effect constraints.

This produces legitimate partial improvements. Customer-held state can reduce data dependence. A replaceable relay can reduce service dependence. An independently controlled authorization gate can constrain certain requests. None makes unavailable source inspectable or converts finite behavioral tests into a proof of all internal behavior.

Re-realization can advance further. An independently specified capability model can become the target for a new implementation, using lawfully available information and materials. Source availability, analysis rights, licensing, redistribution, and access to protected interfaces remain separate considerations. This architecture does not confer rights to copy proprietary code, assets, or secrets, and it

does not assume that every closed system can be reproduced from observations alone.

The distinction matters for commercial adoption. A vendor may offer an inspectable model, reproducible customer builds, exportable state, and independent checking while selling operation and support. Another may permit only constrained external use. Both profiles can be represented honestly, but neither should receive stronger sovereignty claims than its evidence and practical control justify.

The goal is therefore not to relabel opaque software "sovereign." It is to make reliance governable, identify its exact limits, and create a credible route toward replacing the parts that remain unacceptable. Greater openness is valuable when it transfers usable control, not merely when it changes a marketing label.

10. An encrypted-messaging sovereignty exercise

10.1 Define the preserved capability

A hypothetical successor to the originating chat application provides a useful research vehicle. It is not a report of that application's architecture or a recommendation to invent new cryptography. A selected, independently examined protocol implementation would remain a specific dependency. A standardized protocol such as MLS illustrates why authentication, delivery, membership, metadata, and endpoint assumptions must be distinguished from one another [10].

The governing model identifies participants, devices, conversations, memberships, message records, local storage, transport roles, and permitted administration. The contract specifies which payloads may leave which endpoints, how recipients are identified, which diagnostics are prohibited, what state can be retained or exported, and which operations require a change in authorization. Desired offline behavior includes reading retained history and composing queued messages; it does not falsely promise remote delivery without communication.

The user-facing requirement "private messaging" is decomposed into claims. Payload confidentiality against a relay is different from endpoint integrity, metadata minimization, contact-directory privacy, and long-term reconstructability. A failure in one dimension must not be concealed by success in another.

10.2 Admit dependencies by role and effect

A candidate implementation can retain a familiar framework if its relevant source, construction, execution, and effect requirements are satisfied. Sovereignty does not require replacing React merely because it is React, or preferring C merely because it is lower level. The candidate's evidence and control profile decide whether it fits.

The build exercise inventories package installation scripts, generators, transitive

dependencies, assets, and network acquisition. Construction occurs from retained materials under a profile that rejects undeclared fetches. Runtime compartments separate key-bearing operations, message storage, rendering, and transport where the implementation supports adequate isolation. Each compartment receives only the resources its role requires.

A diagnostic feature is either excluded or governed as an explicit opt-in operation with defined data and destination constraints. Removing it requires checking shared initialization and error-handling dependencies; deleting an apparent telemetry function is not proof that all observation paths have disappeared. Likewise, a successful external network-denial test does not establish the absence of misuse through the intended transport.

10.3 Challenge the boundary, not just the feature

Tests should attempt unauthorized endpoint contact, diagnostic access to message content, unapproved dependency acquisition, forged authorization, stale update metadata, altered build material, and mismatches between reviewed and executed artifacts. Positive cases verify that legitimate messages and recovery operations still work. An implementation that prevents leakage by refusing all use is not a successful realization.

The test harness needs its own limited authority and custody. It should not upload private messages while testing whether the application uploads private messages. Synthetic fixtures and deliberately bounded observations can establish the initial campaign. Any later study involving actual communications requires the affected participants' permission and a separate handling policy.

If a proof instrument or compiler is found defective, the qualification system should identify dependent claims and withhold unsupported reuse. SIER supplies a project pattern for per-edge impact and recovery, not a guarantee that every possible dependency has already been modeled [25]. A repaired candidate must not award itself renewed standing.

10.4 Demonstrate continuation and replacement

A second realization or deployment should be prepared under the same declared messaging contract and migration relation. The exercise removes the original hosting service, build environment, and selected registry dependencies. It then reconstructs the application from retained materials, verifies the build and relevant properties, and resumes the specified workflows.

Transport substitution does not automatically preserve identity. The migration must carry verifiable relationships between old and new service addresses, participant credentials, and conversation state. It must not silently promote a server's account identifier into the user's ultimate identity. If the protocol requires an authenticated reinitialization rather than continuation of an old cryptographic epoch, that distinction must be explicit.

Successful restoration demonstrates a bounded exit capability. It does not prove that unavailable hardware can be replaced instantaneously or that every historical secret should remain recoverable. An unsupported step becomes a precise work item: missing source, nonportable state, unavailable key operation, unmatched schema, or absent correspondence evidence.

10.5 Make the outcome legible

An ordinary user should receive a concise view: which communications occur, who controls updates and keys, what works offline, when reconstruction was last tested, and which dependencies still require outside cooperation. A drill-down can expose exact evidence for specialists.

The intended improvement is that the user can answer operational questions without personally auditing every line of code. Delegated experts and agents assist, but their conclusions remain attributable, inspectable, and replaceable. The user gains a usable route to control, rather than being handed a repository and responsibility for an entire software supply chain.

11. Agentic realization and the sovereign commons

The structured-abundance paper proposes agents that explore competing realizations under stable semantic contracts and independently evaluated obligations [1]. This mechanism can turn sovereignty gaps into bounded engineering work. An agent can seek a locally buildable dependency, propose a relay adapter, construct migration tooling, or compare alternative storage mechanisms without being allowed to alter the governing requirement merely to make its candidate pass.

Inference State Images can make repeated, workflow-specific model conditioning reusable where the model and runtime support the required state compatibility. They do not make model behavior authoritative. An ISI's weights, tokenizer, prefix, runtime, provenance, and reconstruction path are themselves dependencies. A sovereign workflow must identify whether a model provider is optional acceleration or an indispensable participant in construction, maintenance, or verification.

Private source and state should not be sent to a remote coding service under an implicit assumption that code generation is harmless. Data disclosure is an effect requiring its own permission. Local models, human maintainers, deterministic tools, and alternative services can occupy different roles. Model portability concerns explicit workflow contracts and practical substitution; it does not imply that KV states are interchangeable across unrelated models.

DKN can distribute models, implementations, build recipes, hardware observations, and evidence without becoming their adopters' authority. Semantic search locates candidates; exact identities and qualified correspondence establish where they fit. Publication rights, local data handling, and current adoption decisions

remain distinct. A remote report can inform a locally authorized quarantine without remotely commanding installation or erasure.

This produces a useful inversion: deeper sovereignty can make external collaboration easier. Contributors can share a better allocator, a protocol model, or a recovery witness without requiring every recipient to adopt their whole system. Reliability work, driver support, interface refinement, and maintenance can participate in the same commons. Their difficulty does not grant permanent proprietary status, although physical experiments and specialist resources still need to be provided.

Independence is not a demand for every user to rebuild everything personally. It is a demand that shared construction and maintenance remain examinable, transferable, and available through more than an unchallengeable relationship. The commons can make those alternatives economically feasible, while local governance prevents the commons itself from becoming a compulsory controller.

12. Authority, shared systems, and ordinary people

12.1 Sovereignty is not administrator absolutism

In a shared system there is no single legitimate actor entitled to every decision. The person who owns a server may not have authority to inspect all communications it transports. Employers, employees, collaborators, maintainers, and data subjects can have distinct rights and responsibilities. A constitutional model must express those relationships rather than equating technical root access with moral or legal legitimacy.

Jurisdiction therefore includes decision procedures: individual consent, group agreement, delegated administration, institutional review, or another legitimate arrangement appropriate to the system. Conflict can require refusal or separation into distinct deployments. No program proves the justice of its governing constitution merely by enforcing it consistently.

Federation can preserve local autonomy while permitting cooperation. Participants negotiate shared operation and protocol contracts; they need not share all policies or trust the same verification providers. Refusal to participate in one federation is distinct from the power to stop another participant's local computation. Interoperability permits interaction under agreed conditions, not a right to compel someone else's service.

12.2 Usable delegation and recoverable administration

A requirement for constant technical supervision would confine sovereignty to specialists. Users need comprehensible defaults, bounded delegation, independent maintenance options, and practiced recovery. Permission interfaces should expose consequential changes in behavior and control, not bury them inside repeated consent prompts.

Recovery deserves the same precision as authorization. Lost keys, unavailable administrators, and disputed succession must not be resolved by an undocumented universal backdoor. The system should offer explicit recovery arrangements whose powers and limits are understood in advance. More recoverability can create additional access paths; a sovereign profile makes that tradeoff visible rather than promising maximum confidentiality and effortless recovery without conditions.

The same principle applies to public evidence. A user should be able to obtain independent help without being forced to publish private state. A provider can sell curation and support, but users should retain an intelligible means to change providers. This is effective delegation, not the fiction that an ordinary person independently evaluates every underlying proof.

12.3 Economic purpose

Structured abundance is valuable here because rights to inspect, fork, and replace are more usable when competent alternatives are affordable. Lower costs of discovery, adaptation, and assurance can reduce dependence on exclusive implementations. Reliability and support can also be collaboratively supplied; they need not remain captive to the original vendor [1].

The commercial role can shift toward delivering a trusted composition, maintaining it, supplying infrastructure, and accepting accountable service obligations. This shift is conditional. A dominant model registry, mandatory signing authority, or private verifier can reintroduce control at a different layer. The relevant success measure is not the number of open packages, but whether users can actually govern and replace what they rely on.

13. Existing evidence and a falsifiable evaluation program

13.1 What the internal artifacts support

The SOV-0 implementation report provides a bounded precedent for the proposed profile discipline. It records thirteen axes, claim-specific support or refusal, explicit opaque regions, reliance warrants, and reconstruction requirements. It reports 112 of 112 court cases and 52 of 52 package tests passing, including 75 typed refusals [2]. These are report statements, not new independently reproduced results.

The CAD-0 report describes a local workbench presenting these custody and reliance distinctions. It reports 96 of 96 court cases and 52 of 52 tests passing, a read-only browser surface, and a governed proposal/review/emission sequence [3]. The report specifically withholds production service safety, multi-user isolation, internal transparency of opaque platforms, automatic activation, and standing increase. Its inherited-artifact checks are not presented as a new complete replay of every predecessor.

These artifacts support the feasibility of representing and exposing sovereignty claims. They do not establish independent reconstruction of a general operating system, removal of every opaque dependency, or reduced exit costs for ordinary users. The critical next step is testing whether profiles correspond to exercised control rather than descriptive compliance.

13.2 Comparative evaluation

A fair evaluation should compare a well-maintained open-source baseline with documented dependencies, ordinary security controls, backups, and competent administration; an improved baseline adding retained build materials and regular restore tests; and the proposed constitutional integration. It should not compare the latter only with an intentionally careless proprietary deployment.

Tasks should include private messaging, a bounded business workflow, and a hardware-facing capability where identified devices are available. Where possible, hold implementations and resources constant to isolate the contribution of governance and evidence interfaces. Report the initial modeling expense separately from later reuse. Compare native and adapted harnesses by tested boundary coverage rather than labels.

Measurements should include successful independent reconstruction, semantic preservation after migration, unauthorized-effect detection, false-refusal rate, time and human work required to change a dependency, completeness of recovered state, and unresolved reliance. Source bytes, contributor counts, and proof-file counts are not substitutes for those outcomes.

13.3 Proposed disruption and migration tests

- **Test:** Provider removal
  Required observation: Required workflows continue or migrate within the declared envelope
  Evidence against the sovereignty claim: Hidden login, activation, or hosted-build requirement blocks continuation
- **Test:** Dependency-source loss
  Required observation: Retained closure reconstructs specified artifacts
  Evidence against the sovereignty claim: A pinned but unretained artifact must be retrieved from the lost source
- **Test:** Unauthorized update
  Required observation: New material cannot become active without applicable local authorization
  Evidence against the sovereignty claim: Authentic vendor publication silently changes code or governing meaning
- **Test:** Telemetry challenge
  Required observation: Disallowed data flow is prevented within the stated observation and enforcement scope

Evidence against the sovereignty claim: Another process or permitted channel bypasses the claimed boundary

- **Test:** Independent restore
  Required observation: A separate operator reconstructs state and required behavior
  Evidence against the sovereignty claim: Export exists but essential meaning, identity, or operation is lost
- **Test:** Checker defect
  Required observation: Affected claims are identified without automatic requalification
  Evidence against the sovereignty claim: Old evidence remains accepted through an incomplete dependency account
- **Test:** Ordinary-user delegation
  Required observation: Users can understand, transfer, and recover administration
  Evidence against the sovereignty claim: Expert dependence is merely relocated to one irreplaceable operator

These are acceptance targets, not completed experiments. Provider removal and fault injection must occur in an authorized test environment. Effects on real collaborators, devices, or production data require additional scope and safeguards.

13.4 Falsifiable hypotheses

The central technical hypothesis is that explicit custody, effect, and reconstruction contracts reduce undiscovered dependence at matched engineering effort. It is weakened if the model merely restates configuration while bypasses persist. A second hypothesis is that reusable realization and evidence packages reduce the marginal cost of exit. It is weakened if semantic reconciliation or state migration must be repeated almost entirely for each adopter.

A third hypothesis concerns usable sovereignty: guided profiles and transferable delegation should let nonspecialists exercise meaningful decisions without equivalent increases in error or administrative burden. It is weakened if users routinely misinterpret evidence, disable essential protections, or become unable to recover independently of a new intermediary.

Success need not mean every claim improves. Componentwise comparisons may reveal greater runtime control but higher recovery burden. Publishing such tradeoffs would be more informative than compressing them into a promotional score.

13.5 Universal GSM and Realization-Funnel Placement

Sovereign software is the full-substrate custody and exit face of Constitutional Computing. Within the universal **Governed Semantic Model (GSM)**, its central question is not whether a product carries a sovereignty label, but whether a typed claim about control can be traced through principals, capabilities, dependencies, authority, evidence, and continuation conditions. The **Governed Meaning Base (GMB)** is the current name in the author-confirmed `CCF -> CMB -> GMB` lineage. A GMB can preserve this model and its revision history; storing the model does not itself establish custody of the described software or authority over it.

The complete realization funnel sharpens where sovereignty can be lost:

$$\begin{aligned} & P \\ & \to S \\ & \to CM_1 \\ & \to CM_2 \\ & \to IC \\ & \to IR \\ & \to E \\ & \to R \end{aligned} \quad .$$

Possibility contains candidate capability arrangements, dependency substitutions, governance topologies, and exit paths. **Specification** states the preserved capability, legitimate principals, required invariants, forbidden effects, authority boundaries, and evidence obligations. **Configuration Manifold 1 (CM1)** contains configurations satisfying mandatory admissibility, including required rights, custody, security, and reconstruction constraints. **Configuration Manifold 2 (CM2)** selects the preferable admissible region under declared tradeoffs such as inspectability, cost, accessibility, performance, maintenance burden, decentralization, and resilience. **IC** may be expanded interchangeably as **Instance Constitution** or **Instance Configuration**; it binds one concrete dependency, authority, update, evidence, and exit profile. **IR** may be expanded interchangeably as **Instance Realization** or **Implementation Realization**; it is the actual built and deployed system. **Evidence** evaluates claim-relative control, correspondence, effects, and continuation. **Revision** responds to newly discovered dependencies, compromised tools, changed rights, provider withdrawal, new vulnerabilities, and failed drills.

The funnel prevents four recurrent collapses. A possible replacement is not yet admissible. An admissible alternative is not necessarily preferred. A preferred configuration is not necessarily selected. A selected constitution is not necessarily realized or evidenced. Conversely, a running realization does not acquire authority to redefine the specification that it happens to implement.

13.6 Semantic Kernel: Primitives, Invariants, and Adjacencies

The paper's compact primitive set is: **principal**, **jurisdiction**, **capability**, **semantic contract**, **dependency node**, **typed dependency edge**, **authority grant**, **custody profile**, **reliance record**, **evidence object**, **exit contract**, **exit witness**, **configuration**, **realization**, and **revision event**. The thirteen custody dimensions refine the custody-profile primitive; they are not thirteen independent sovereignty scores.

The integration yields the following invariants:

1. A sovereignty claim cannot exceed the least-supported dependency relationship required for that claim.
2. Inventory does not imply control; source possession does not imply reconstructability; export does not imply continuation.
3. Evidence about an implementation does not grant authority to adopt, execute, update, or replace it.
4. A realization may not silently redefine the capability contract, principal, jurisdiction, or preserved meaning that qualifies it.
5. Opaque reliance must remain typed as opaque; behavioral evidence cannot be promoted into knowledge of unobserved internals.
6. A provider-exit claim requires an exercised witness under a declared resource and dependency-removal envelope.
7. Shared verification, model, registry, signing, or governance machinery is itself part of the dependency closure when it is indispensable.
8. CM2 preference may not convert a mandatory right, safety constraint, or authority boundary into a compensable objective.
9. IC and IR name one selection and realization pair even when their accepted expansions vary: Instance Constitution / Instance Configuration and Instance Realization / Implementation Realization.
10. Revision must identify which earlier claims and evidence become stale when a dependency, authority, contract, or realization changes.

Its strongest corpus adjacencies are functional rather than synonymous. *Constitutional Computing: Structured Abundance* explains how multiple qualified realizations and shared assurance may lower substitution cost; this paper states why practical substitution matters. GSOL specifies governed operation contracts. GOORRT separates objects, operations, and realizations. Cognitive ABI makes capability and qualification interfaces portable across components. *Reasoning by Way of Receipts* supplies a method for local, attributable state and claim transitions, while receipts remain evidence rather than sovereignty. GMB preserves the meaning and lineage fabric. DKN distributes candidates and evidence without transferring adoption authority. SIER traces the impact of defective evidence through dependent claims. None of these components individually supplies full-substrate sovereignty.

13.7 Integration Countertests

The integrated thesis should be weakened or revised if any of the following holds: (1) the thirteen-axis profile cannot predict failure in a provider-removal drill better than an ordinary dependency inventory; (2) teams cannot agree on claim-relative closure rules without effectively modeling the entire world; (3) a retained GMB cannot be interpreted independently of one provider; (4) semantic preservation cannot be tested across heterogeneous realizations; (5) exit exercises routinely destroy security properties or legitimate shared authority; (6) the additional governance layer costs more than the substitution capacity it creates; (7) ordinary users cannot distinguish evidence, authority, and practical control; (8) ostensibly independent realizations share an undisclosed critical dependency; (9) the verifier or agent infrastructure becomes a new indispensable controller; (10) local adoption authority blocks necessary coordinated safety or interoperability; (11) the profile hides political and economic dependence behind technical custody; or (12) exercise success does not survive meaningful time, scale, or personnel turnover.

13.9 Corpus Integration Sources

The integration additions above were checked against exact local heads: *Constitutional Computing: Structured Abundance* (SHA-256 `A1EF0D1F7B5DF84A107E986421EAA95827AEDCB801B7DA63F72BEECA6047BE8D`); *A Cognitive ABI for Capability-Centered Computing* (

`F7A7034BB9E382C91AEA693BD656040A93C5C5B504FE70425A31561CEECDC084`); *Governed Semantic Operation Language* (
`8B0639DC078F7536F143F40DAC65DCC86AFAA551EC98581E97A2886B094C4C0A`); GOORRT (
`152AB6781AA827519858E0BD8A1811CB1F2C08F56599CD2A61896115742D1399`); *Reasoning by Way of Receipts* (
`8D884B96597EA6303A324173A7CE2D8C735A952E91FBE4C9EB6509FA6D6B195F`); and the GMB semantic-manifold document (
`49A05A424F226A21C81D638727E1165DE1AD55AB1CBDC56EA65D67A1362A4B08`). These bindings establish comparison custody, not adoption, identity merger, or truth of every proposition in those works.

Sovereignty as Effective Continuity Across a Dependency Closure

Across the completed P1 corpus, Sovereign Software is best compressed to a **measurable relation between an actor or community, a capability, and its dependency closure under a declared threat model**. The relation concerns effective inspectability, governance, reconstruction, delegation, replacement, and exit. It is not a product label, source-code possession, administrator absolutism, isolation, or freedom from all dependence.

- **Adjacent work:** *Jnana and the Next Open Source Revolution*
  Supplies: A wider collaborative and constitutional vision for open software contribution.
  Sovereign-software boundary: Sovereign Software supplies the dependency-control and exit criteria; it is not the whole social program.
- **Adjacent work:** *Model-First Legacy Migration - A Capability Thesis*
  Supplies: Capability-preserving reconstruction across legacy boundaries.
  Sovereign-software boundary: Migration can realize one sovereignty test; successful translation does not establish full dependency control.
- **Adjacent work:** *What Software Systems Actually Receipt, Govern, and Hide*
  Supplies: Analysis of visibility, governed surfaces, and hidden operations.
  Sovereign-software boundary: Observability is one dimension of sovereignty, not its entirety.
- **Adjacent work:** *Testing for Deriving Observable State of Software Systems*
  Supplies: Tests that derive claims about system state from bounded observations.
  Sovereign-software boundary: Derived state supports assurance; it does not by itself create authority, rebuildability, or exit.
- **Adjacent work:** *Universal Governance Kernel (UGK)*
  Supplies: A candidate reusable mechanism for governed transitions and effects.
  Sovereign-software boundary: A kernel can mediate part of the closure; it remains inside the closure and subject to the same sovereignty tests.

For actor or community a, preserved capability c, system closure S, time t, and threat model $Theta$, record a vector:

$$Sov(a, c, S, t, Theta) = (inspect, modify, build, run, observe, govern, delegate, replace, reconstruct, exit).$$

Each component needs an effective, not merely formal, test. Published source without reproducible construction is not build sovereignty. A configurable interface without inspectable hidden services is not governance sovereignty. Data export without a working provider-removal exercise is not effective exit. Root access without safe delegation, recovery, and community continuity may produce domination rather than sovereignty.

Sovereignty is capability- and actor-relative. One user may have practical exit while another lacks the resources, accessibility, keys, knowledge, or legal permission to exercise the nominal option. Shared institutions can increase sovereignty by making difficult powers collectively available, provided delegation remains inspectable, revocable, recoverable, and non-dominating.

The universal GSM can type `DEPENDS_ON_CLOSURE`, `INSPECTABLE_BY`, `BUILDABLE_BY`,
`OPERABLE_BY`, `GOVERNED_BY`, `DELEGATED_TO`, `REVOKABLE_BY`, `REPLACEABLE_BY`,
`RECONSTRUCTABLE_BY`, `EXITS_FROM`, `DOMINATED_BY`,
and
`TESTED_BY_DISRUPTION`. Evidence must bind the exact actor, capability, closure, environment, time, and threat model.

This integration establishes four editorial commitments: adopt effective continuity across a dependency closure as the official corpus role; adopt the sovereignty vector; require actor-, capability-, time-, and threat-model-relative evidence; and make provider removal plus reconstruction the strongest ordinary exit test rather than source availability alone.

Sovereignty Evidence Under Adversarial Calibration and Structural Warrant

The fixed-seed comparison with adversarial semantic pressure, Proxy Collapse, A Constitution for the Context Window, and Warrant by Structure yields a claim-level sovereignty case:

$$\begin{aligned} S^* = (&actor, capability, closure, threat_model,\\ &sovereignty_dimension, test, proxy_assumptions,\\ &authority_path, effect_evidence, warrant, expiry,\\ &reconstruction) \end{aligned}.$$

Adversarial analysis challenges hidden dependencies, compromised witnesses, staged observations, captured updates, and formally valid but misleading receipts. Proxy analysis asks whether source availability, export, local control, or successful replay still predicts the claimed inspectability, reconstruction, governance, or exit capability in the current environment.

Constitutional context compilation is part of the closure whenever an agent's cognition or action depends on retrieval, memory, policy, tools, adapters, providers, or hidden transformation. Sovereignty requires inspectable and replaceable context-building obligations proportionate to the claim; it does not require publishing protected user content.

Structural warrant evaluates whether the exact sovereignty claim is supported by legitimate authority, mandatory paths, independent-enough witnesses, actual effect observations, bypass analysis, and a declared ceiling. No receipt, build, export, or provider-removal exercise warrants dimensions it did not test.

The universal GSM can type
`CLAIMS_SOVEREIGNTY_DIMENSION`,
`CHALLENGED_BY_ADVERSARIAL_CLOSURE`, `USES_EXIT_PROXY`, `GOVERNS_CONTEXT_DEPENDENCY`,
`WARRANTED_BY_DISRUPTION_TEST`, `EXPIRES_WITH_CLOSURE_CHANGE`,

and
`RECONSTRUCTS_CAPABILITY_INDEPENDENTLY`.

This randomized review establishes four editorial commitments: adopt the claim-level sovereignty case; red-team proxy and witness assumptions; include context compilation in relevant closures; and require structural warrant and expiry for every asserted sovereignty dimension.

Claim-Relative Sovereignty and Exercised Exit

Sixteen retained downstream uses apply Sovereign Software to context control, capability substitution, communal and collective computing, cold boot, adversarial trust, monetary infrastructure, constitutional development, incentive design, open-source collaboration, legacy migration, operating systems, proxy testing, project lineage, semantic fragmentation, and structural warrant. They converge on one point: sovereignty is not a product property or a scalar score. It is a principal-, capability-, jurisdiction-, and time-relative claim about non-domination, practical control, and recoverable continuation across a dependency closure.

The claim envelope is:

$$\begin{aligned} S^* = (&principal, capability, jurisdiction, time,\\ &dependency_closure, custody_profile, rights,\\ &delegations, opaque_reliance, alternative_set,\\ &resource_envelope, exit_plan, exercise, witnesses,\\ &correlated_failures, remedy, expiry, remainder) \end{aligned} \quad .$$

Technical possession, legal permission, institutional legitimacy, usable skill, economic feasibility, and social acceptability are distinct dimensions. A system may be open source yet practically unreconstructable, locally runnable yet institutionally illegitimate, forkable yet unable to preserve a shared service, or replaceable only by alternatives sharing the same provider, key, registry, model, build chain, or governance dependency. The least-supported required dimension bounds only the exact sovereignty claim that depends on it.

Exit evidence must be exercised. A provider-removal or reconstruction trial binds the preserved capability, starting state, excluded provider set, resource and time envelope, substitute dependencies, security properties, data and semantic migration, affected parties, observed result, and unresolved remainder. A successful drill is time-bounded evidence, not permanent independence. Delegation remains compatible with sovereignty when the principal can inspect the grant, understand its consequence, revoke or replace it under the governing arrangement, and recover from failure without acquiring forbidden authority over others.

The universal GSM can type
`CLAIMS_SOVEREIGNTY_FOR_CAPABILITY`,

DEPENDS_ON_SOVEREIGNTY_DIMENSION, DELEGATES_WITH_RECOVERY, EXERCISES_PROVIDER_EXIT, SHARES_CORRELATED_FAILURE_DOMAIN, PRESERVES_SHARED_AUTHORITY_ON_EXIT, EXPIRES_SOVEREIGNTY_EVIDENCE, and RETAINS_OPAQUE_RELIANCE.

This reverse-dependency review establishes four editorial commitments: adopt the claim-relative sovereignty envelope; distinguish technical, legal, institutional, practical, economic, and social dimensions; require exercised exit under a declared envelope; and treat delegation, shared authority, correlated alternatives, and opaque reliance as explicit constraints rather than automatic failures or successes.

Sovereignty Must Survive an Exercised Exit

Public orientation

Sovereign Software asks whether a person can inspect, reconstruct, modify, replace, and continue using software without dependence becoming subordination. Source availability is one input. Effective sovereignty extends through dependency closure, build and execution authority, data, keys, services, updates, hardware boundaries, and the practical ability to exit.

Adversarial tests

1. **Source-only sovereignty.** Code is available but dependencies, toolchains, models, assets, schemas, or services are missing. Reconstruction must cover the declared closure.
2. **Reproducibility-without-usability.** A build reproduces but cannot be configured, operated, repaired, or migrated by the user. Byte identity is not effective control.
3. **Service-hostage attack.** Local code depends on an irreplaceable remote API, identity provider, update channel, or data service. Substitution and offline degradation must be explicit.
4. **Key-and-certificate capture.** The user can build code but cannot authorize execution, sign updates, decrypt data, or replace trust roots. Custody of authority artifacts is part of sovereignty.
5. **Hardware-boundary attack.** Firmware, secure boot, drivers, accelerators, or proprietary interfaces prevent inspection or replacement. The claimed sovereignty boundary must stop where evidence stops.
6. **Exit-cost attack.** Export technically exists but time, expertise, compute, legal terms, or format lock-in make exit impracticable. Effective exit requires measured burden.
7. **Update-subordination attack.** Security maintenance depends on the provider even after nominal forkability. Continuity needs a maintenance and vulnerability-response story.
8. **Sovereignty-centralization attack.** A community replaces one vendor with an unaccountable maintainer or foundation. Plural governance, fork rights, and dependency transparency still matter.

Claim and evidence ceiling

The paper supplies a sovereignty model and evaluation program. It does not prove that a named system is fully sovereign without an exercised reconstruction, provider-removal, key-transfer, data-export, offline-operation, repair, and successor-maintenance demonstration across its declared boundary.

Claims about software freedom history, planned obsolescence, economic accessibility, security, sustainability, or social outcomes require external evidence. Internal corpus systems show possible realization paths but do not themselves prove adoption or user benefit. No external research was added in this pass.

Reader-facing consequence

The public standard is concrete: if the original provider disappears or becomes unacceptable, can the user preserve their data and meaning, reconstruct the system, authorize it to run, replace dependencies, continue maintenance, and understand the remaining boundary?

14. Limits, failure modes, and research priorities

The proposed architecture can fail through incomplete models, defective encodings, compromised enforcement, unavailable observations, misunderstood rights, and impractical reconstruction. It can also succeed technically while concentrating institutional power. Public source and elaborate receipts do not prevent a dominant authority from controlling which interpretations or implementations are permitted inside a closed marketplace.

Three forms of confusion deserve particular attention. **Opacity laundering** occurs when external tests are presented as knowledge of internals. **Authority laundering** occurs when a build, signature, or machine finding is treated as permission. **Exit laundering** occurs when an export or nominal alternative is presented as independently usable continuation. Claim-specific evidence and exercised disruption tests are proposed countermeasures, not guarantees that the confusions cannot recur.

The recursion toward a full substrate must stop, for each particular claim, at explicit assumptions and observation boundaries. That stopping point is not a declaration that deeper sovereignty is impossible. It identifies work still required: another independently buildable component, a stronger hardware model, an additional witness, or a different deployment. A claim about software confinement cannot quietly become a claim about all physical side channels, and a declaration of trust is not evidence that the trust is justified.

The priority is therefore a sequence of experimentally defensible expansions rather than a universal sovereignty label. Improve one critical boundary, preserve its evidence, test that successors can reconstruct it, and expose what remains. Distributed collaboration can make those improvements cumulative without assuming that more contributors automatically provide independent assurance.

15. Conclusion: dependence without subordination

Sovereign software is not established merely by possessing code, controlling a server, using encryption, or receiving an attestation. It requires a practical relationship between legitimate authority and the full set of dependencies relevant to the claims a user needs to make. Meaning, materials, effects, updates, verification, and continuation must remain governable rather than becoming powers silently delegated to an indispensable provider.

Constitutional Computing offers a proposed means to sustain that relationship. Explicit semantic contracts keep capabilities distinct from their implementations. Custody profiles reveal control and opacity. Independent evidence limits claims. Governed execution constrains effects. Reconstruction exercises distinguish nominal portability from effective exit. Agents and distributed realization libraries can make replacement and maintenance collaborative rather than requiring every user to become the sole author of an entire computing stack.

The two companion papers therefore address different parts of one program. Structured abundance concerns how qualified capability can accumulate and become easier to reuse. Sovereign software concerns who retains control when that capability is adopted, changed, or withdrawn. Abundance makes sovereignty more practical; sovereignty prevents abundance from becoming another mechanism of capture.

The objective is not isolation from everyone else's work. It is the capacity to benefit from that work without surrendering the ability to understand, constrain, refuse, continue, and replace it. **Owning the full substrate means making the machinery of dependence answerable to those who legitimately rely on it.**

Editorial Provenance and Standing

Source, Evidence, and Standing

This is an AI-prepared successor revision candidate derived from one exact author manuscript. The integration custody manifest is `work/paper-saturation-v1/evidence/sovereign-software/integration-custody-manifest.v0.1.json`, SHA-256 `7D6D1ABC0BB8C1B8BE990ABF946BD665218D75B3CCB9B43771E030A3A436448C`. The exact source remains unchanged. The definition of sovereign software, thirteen custody dimensions, dependency-closure model, provider-removal exercise, encrypted-messaging scenario, structured-abundance relationship, evaluation program, limits, conclusion, and original references are author material. This pass standardizes the Ableman paper header and Pandoc math,

places the argument in the universal GSM and complete realization funnel, adds a compact primitive/invariant/adjacency map, and exposes open editorial questions. Those additions are AI editorial or synthetic contributions until adopted. The manuscript already distinguishes internal report statements from independently reproduced findings and describes its exercises as proposed research instruments. This pass preserves that discipline. It does not bind the exact SOV-0, CAD-0, or SIER report files; replay their reported results; audit the originating application; or revalidate external sources. No sovereignty, system conformance, publication, canonical standing, identity merger, or implementation authority is granted.

Corpus-Wide Reference Positioning

This paper argues that possessing application source does not by itself establish control over the system that gives the application its behavior. Its nearest external comparison fields are open sovereign, security, software. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Source availability and reproducible builds improve inspectability but do not alone guarantee comprehensibility, practical exit, hardware control, or sustainable maintenance. Unforgeable capabilities can constrain exercise of authority while still leaving allocation, delegation, revocation, and social legitimacy incorrectly specified. Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Free Software Foundation. *The Free Software Definition.* https://www.gnu.org/philosophy/free-sw.html **Relationship:** Normative predecessor for user freedoms; the paper may add reconstruction, governance, or hardware-boundary obligations beyond license rights.

[**E2**] Open Source Initiative. *The Open Source Definition.* https://opensource.org/osd **Relationship:** Adjacent licensing baseline; source availability is not treated as sufficient for operational sovereignty.

[**E3**] Martin Kleppmann et al. "Local-First Software: You Own Your Data, in Spite of the Cloud." *Onward! 2019*, pp. 154-178. DOI: 10.1145/3359591.3359737. https://www.inkandswitch.com/local-first/ **Relationship:** Adjacent local-first design program relevant to user custody, offline operation, and collaboration.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds.* reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] Jerome H. Saltzer and Michael D. Schroeder. "The Protection of Information in Computer Systems." *Proceedings of the IEEE* 63(9), 1975, pp. 1278-1308. DOI: 10.1109/PROC.1975.9939. https://doi.org/10.1109/PROC.1975.9939 **Relationship:** Foundational protection principles used as a comparison point for least authority, mediation, and inspectability.

[**E6**] Mark S. Miller. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control.* PhD dissertation, Johns Hopkins University, 2006. http://www.erights.org/talks/thesis/ **Relationship:** Predecessor object-capability treatment of authority and compositional security.

[**E7**] Gerwin Klein et al. "seL4: Formal Verification of an OS Kernel." *SOSP '09*, 2009, pp. 207-220. DOI: 10.1145/1629575.1629596. https://doi.org/10.1145/1629575.1629596 **Relationship:** Predecessor for machine-checked implementation correspondence and capability-based kernel design.

Source-head external references retained under original labels [7] GNU Project / Free Software Foundation. (n.d.). *What Is Free Software?* The Free Software Definition. Official explanatory text. Source: gnu.org/philosophy/free-sw.en.html.

[8] Open Source Initiative. (n.d.). *The Open Source Definition*, version 1.9. Official definition. Source: opensource.org/osd.

[9] Kleppmann, M., Wiggins, A., van Hardenberg, P., & McGranaghan, M. (2019). *Local-First Software: You Own Your Data, in Spite of the Cloud*. Ink & Switch essay and Onward! 2019 paper. Source consulted: inkandswitch.com/essay/local-first/.

[10] Barnes, R., Beurdouche, B., Robert, R., Millican, J., Omara, E., & Cohn-Gordon, K. (2023). *The Messaging Layer Security (MLS) Protocol*. RFC 9420. Internet Engineering Task Force. doi:10.17487/RFC9420.

[11] Pohle, J., & Thiel, T. (2020). Digital sovereignty. *Internet Policy Review, 9*(4). doi:10.14763/2020.4.1532.

[12] Saltzer, J. H., & Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE, 63*(9), 1278–1308. Author-hosted text: web.mit.edu/Saltzer/www/publications/protection/.

[13] University of Cambridge Computer Laboratory. (n.d.). *Capsicum: Practical Capabilities for UNIX*. Project description and technical overview. Source: cl.cam.ac.uk/research/security/capsicum/.

[14] Reproducible Builds Project. (n.d.). *Definitions: When Is a Build Reproducible?* Official documentation. Source: reproducible-builds.org/docs/definition/.

[15] Dolstra, E., de Jonge, M., & Visser, E. (2004). Nix: A safe and policy-free system for software deployment. *Proceedings of the 18th Large Installation System Administration Conference*, 79–92. USENIX.

[16] Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., & Cappos, J. (2019). In-toto: Providing farm-to-table guarantees for bits and bytes. *28th USENIX Security Symposium*, 1393–1410. USENIX.

[17] The Update Framework Project. (2026). *The Update Framework Specification*, version 1.0.37, 8 October 2026. Sections 1–2 and client workflow. Source: theupdateframework.github.io/specification/latest/. Version and date identify the consulted edition; a moving endpoint is not an immutable artifact pin.

[18] Wheeler, D. A. (2010). *Fully Countering Trusting Trust through Diverse Double-Compiling*. Dissertation preprint, arXiv:1004.5534.

[19] Bootstrappable Builds Project. (n.d.). *Bootstrappable Builds*. Project overview. Source: bootstrappable.org.

[20] seL4 Project. (n.d.). *Verification*. Official overview, proof implications, and assumptions. Source: sel4.systems/Verification/.

[21] University of Cambridge / SRI International. (n.d.). *Capability Hardware Enhanced RISC Instructions (CHERI)*. Project overview. Source: cl.cam.ac.uk/research/security/ctsrd/cheri/.

[22] SPDX Project. (n.d.). *Overview*. Software bill-of-materials and specification overview. Source: spdx.dev/learn/overview/.

[24] CompCert Project. (n.d.). *CompCert C: A Trustworthy Compiler*. Manual, Chapter 1. Source: compcert.org/man/manual001.html.

Ableman corpus

[**A1**] Adam Ableman. *Jnana*. Ableman Corpus current edition; work identity `AMCS-LOCAL-WORK-ECEAB04A620F163D`;
source SHA-256
`76D921D2975BDCD25D6DFB506F5E85F7AFDBF4D2CCC65F8C45394121F9D90060`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Jnana and the Next Open Source Revolution*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-473AD6FAA1659E24`;
source SHA-256
`E73859FD3B4AD4F7ED80E96F93F2D5A0DCB3A2D76C5741813F878B3874D7887D`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *The Substrate Migration Principle*. Ableman Corpus current edition; work identity `PAPER.PUB.18726868`;
source SHA-256
`94111190A4C08A4E739FF7E77BF7B3C37C959E47A33F21CD1DAA29668C9B4BA0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Cold Boot Continuity as a Deployment Primitive*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-3DB3470715D914D0`;
source SHA-256
`4CBB05E90E4ECA37EDA78C0E1FC1FDD180B2D6E0936831B57379AD7124776D4E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Effect Sovereignty*. Ableman Corpus current edition; work identity `PAPER.PUB.18737323`;
source SHA-256
`1DB8A34946A8705F6F4BA4FC3B12B251FC7C83DBDF217F9DDFF55529C8612EE7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality*. Ableman Corpus current edition; work identity `PAPER.PUB.18601631`;
source SHA-256

6DE46F821594AB11BFB7F8373368C452BD89E5BD4EB172B88D9467E87CF74C33.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[A7] Adam Ableman. *The Integrity Enforcement Layer*. Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-505307E9E6571B63;
source SHA-256
F858AB32E0937B796353CAE573DD4183FDF4F179A3E1E7E40C4E0C6BF1AB0018.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Ableman Corpus references *References [1–6,23,25] are manuscripts or reports from the Constitutional Computing research corpus. They are cited as architecture and bounded project evidence, not as independently replicated findings. External sources were consulted on 18 October 2026. This is a selected literature review, not a systematic novelty or legal analysis.*

[1] Adam Ableman (2026). *Constitutional Computing: Evidence-Carrying Realization, Agentic Development, and the Economics of Structured Abundance*. Research-position paper, 18 October 2026. Companion manuscript. Retained file:
Constitutional_Computing_Structured_

Abundance.md.

[2] Constitutional Computing research corpus. (n.d.). *SOV-0 Implementation Report*. Bounded, nonproduction substrate-custody model report; no standing promotion. Retained file:
GOORRT_SOV0_IMPLEMENTATION_REPORT.md.

[3] Constitutional Computing research corpus. (n.d.). *CAD-0: Local Constitutional Custody Workbench*. Bounded, nonproduction implementation report. Retained file:
GOORRT_CAD0_IMPLEMENTATION_REPORT.md.

[4] Adam Ableman (n.d.). *Governed Semantic Operation Language: A Constitutional Interface Between Prose, Policy, Code, Proof, and Execution*. Ableman Corpus edition. Retained file:
Governed Semantic Operation Language(2).md.

[5] Constitutional Computing research corpus. (n.d.). *GOORRT: The Governed Object–Operation–Realization Representation Triad*. Architecture document. Retained file:
GOORRT- The Governed Object--Operation--

Realization Representation Triad.md.

[6] Adam Ableman (2026). *A Cognitive ABI for Capability-Centered Computing: Evidence-Carrying Composition of Fuzzy and Deterministic Semantic Compilers*.

September 2026. Ableman Corpus edition. Retained file: `Cognitive_ABI_Evidence_Carrying_Composition.md`.

[23] Constitutional Computing research corpus. (2026). *TCLA-A01: Tessera Constitutional Language Architecture*. 14 October 2026. Controlling-amendment candidate, not admitted. Retained file: `01_TCLA_CONTROLLING_AMENDMENT_CANDIDATE.md`.

[25] Constitutional Computing research corpus. (2026). *SIER Q2: Semantic Integrity and Evidence Recovery–Completion Report*. 14 October 2026. Information-only, bounded-evidence report. Retained file: `SIER-Q2-REPORT.md`.

Research, provenance, and artifact statement

This manuscript is a conceptual synthesis and companion paper. The originating encrypted-chat account was supplied by the author on 18 October 2026; it is not a reconstructed audit of that application's source, dependencies, or network behavior. Named technologies identify the development context, not a finding of inherent telemetry or misconduct. The messaging exercise, disruption tests, and exit-cost model are proposed research instruments. No new proofs, builds of the described systems, migrations, hardware tests, or user studies were performed for this manuscript. Existing project results retain their original scope and nonproduction limitations. Preparation used AI-assisted drafting, source retrieval, and document production. This paper neither amends a controlling contract nor authorizes execution, admission, or standing promotion.

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23848227